



CYBERITH **IK**

Unity Plugin – Integration Guideline

Inverse Kinematics system based on Final IK VRIK by



ROOTMOTION

Cyberith GmbH

Teslastraße 6
3100 St. Pölten
Austria
FN 410899p

For any questions, please contact

Cyberith Support
support@cyberith.com
+43 1 890 17 13

Table of Contents

Prerequisites	4
Window 7 or newer	4
Unity Game Engine	4
For compiling C# Code: Visual Studio	4
A Unity Project containing the:	4
Cyberith SDK Core Plugin	4
SteamVR Plugin	4
For extended functionality: FinalIK Plugin for Unity	4
Introduction to CyberithIK	5
Setup Example Project	6
1. Create an empty project containing the Cyberith Core SDK Plugin (CybSDK)	6
2. Import the SteamVR Plugin.....	6
3. Import the CyberithIK Plugin	6
4. Open Example Scene.....	6
5. Add Layers and Tags.....	6
6. Generate Input Bindings for SteamVR.....	7
7. Start your 4-point CyberithIK experience	8
8. Start your 6-point CyberithIK experience	8
Example Project using Microsoft Rocketbox Avatar	9
Setup Your Own Project	10
1. Import the Cyberith Core Plugin for Unity and the CyberithIK Plugin in your project	10
2. Import the SteamVR Plugin.....	10
3. Add Layers and Tags.....	10
4. Open your Scene	11
5. Add CVirtPlayerControllerIK Prefab to Scene	11
6. Start your 4-point CyberithIK experience	11
7. Start your 6-point CyberithIK experience	12
Setup Custom Avatar in Your Own Scene	13
1. Add the Model Files to Your Project	13
2. Insert the Model into the Forward Direction Game Object	13

3.	Copy Components to Avatar Game Object.....	13
4.	Add References in VRIK Component.....	13
5.	Duplicate Skinned Mesh Renderer Game Object (Necessary for Rocketbox)	14
6.	Add the Correct Model Renderers.....	15
7.	Set up C Ik Finger System Component.....	16
8.	Add the References to Your Avatar	18
9.	Disable Cyberith Model Game Object.....	19
10.	Test Your Avatar.....	19
	Documentation	20
	The C Ik System Calibration Script & the Neck Offset	20
	The C IK System Script and the remaining Offsets	22
	The C Ik System Calibrated Model Script: Materials and Mesh Renderers.....	25
	Add-On Feature: The C Ik Finger System Script.....	27
	Inspector Variables	28
	Changing XR Plug-in.....	30
	Experimental: Clipping Plane Shader.....	32

Prerequisites

Window 7 or newer

Unity Game Engine

Compatible: Unity 2020 or newer – <http://unity3d.com/>

For compiling C# Code: Visual Studio

A Unity Project containing the:

[Cyberith SDK Core Plugin](#)

The Cyberith IK Plugin is an extension to the Cyberith SDK Core Plugin for Unity. Please make sure that you have a fully working Unity Project using the Cyberith SDK Core Plugin before continuing.

[SteamVR Plugin](#)

The Cyberith IK System for Unity is currently based on the SteamVR Plugin for Unity (<https://assetstore.unity.com/packages/tools/integration/steamvr-plugin-32647>).

If you need it to work with an alternative system (e.g., OpenXR, etc.) you can change the code on your own discretion.

For extended functionality: FinalIK Plugin for Unity

If you find this IK System useful, please purchase the FinalIK Plugin from the Unity Asset Store: <https://assetstore.unity.com/packages/tools/animation/final-ik-14290>

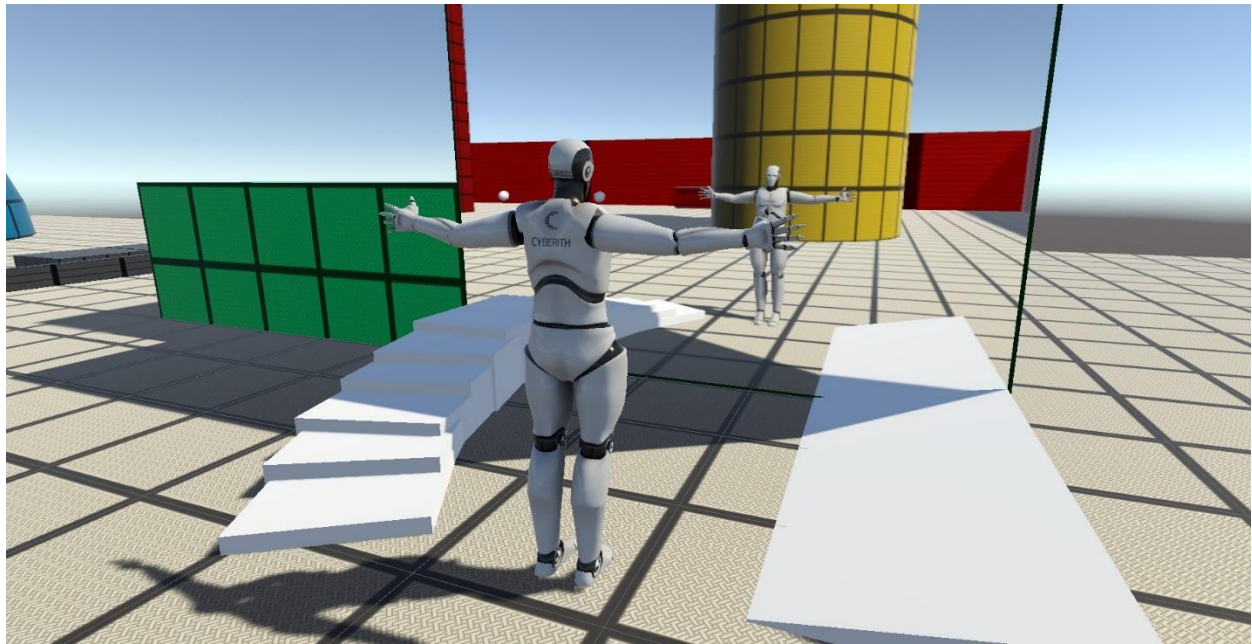
As Cyberith IK is based on FinalIK it is compatible and allows you to use the extended functionality of FinalIK!

Your purchase supports Rootmotion, the developer for FinalIK, the most powerful IK solver for Unity.

Introduction to CyberithIK

Cyberith IK is our approach to a user friendly, powerful inverse kinematics (IK) system for realistically moving avatars in VR.

The Cyberith IK system is based on Rootmotion's FinalIK, the most powerful IK solver for Unity.



Please go through the “Cyberith IK – Basics & General Information” instructions document before continuing with this document.

The “Cyberith IK – Basics & General Information” instructions document provides key information that you should definitely know before starting to implement Cyberith IK in your Unity Project!

Setup Example Project

After testing CyberithIK with our demo application, you can set up the CyberithIK in a Unity project. Setting up the provided example map is done by the following steps:

1. Create an empty project containing the Cyberith Core SDK Plugin (CybSDK)

At the time you are starting to implement CyberithIK, you should be familiar with the Cyberith Core SDK (CybSDK) Plugin for Unity already. If you are not familiar with the CybSDK yet, please go through the applicable documentation, [the Unity Core Integration Guideline](#).

2. Import the SteamVR Plugin

CyberithIK is dependent on the SteamVR Plugin. Please import the SteamVR Plugin, that you can find (for free) on the Unity Asset Store: <https://assetstore.unity.com/packages/tools/integration/steamvr-plugin-32647>

3. Import the CyberithIK Plugin

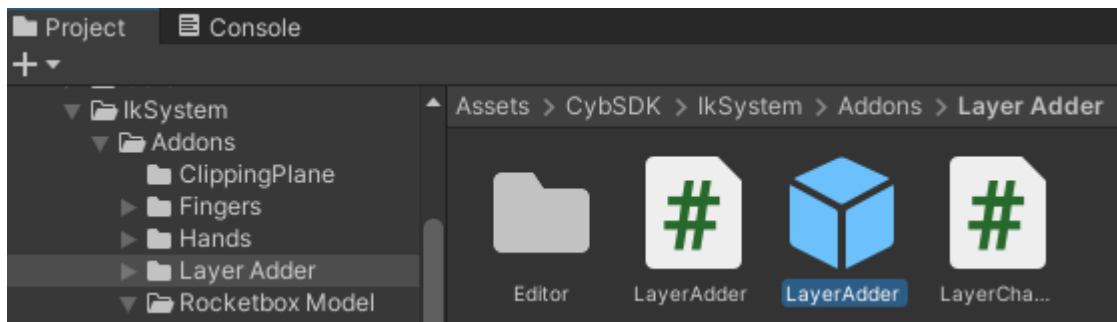
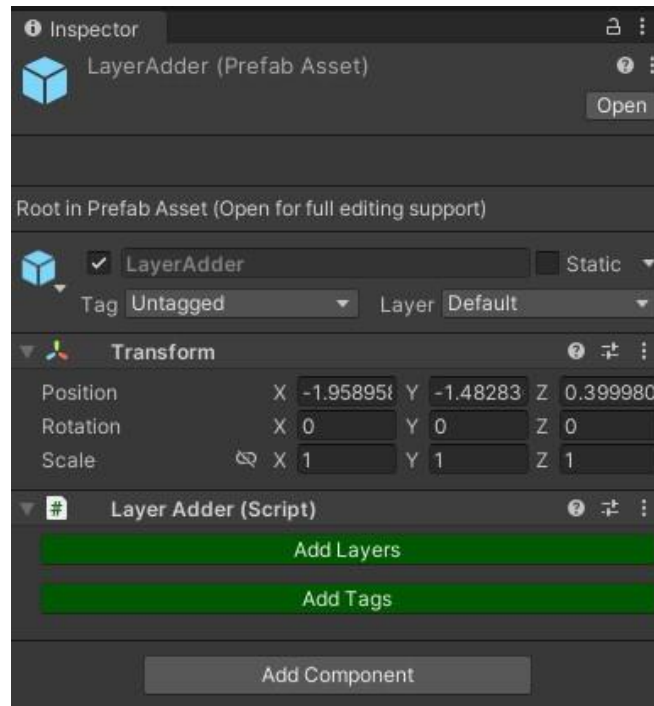
CyberithIK comes as a separate plugin for Unity, which requires the Cyberith Core SDK Plugin to be imported. Import the plugin through the menu “Assets → Import Package → Custom Package ...” or double-click the package in the File Explorer when your project is open.

4. Open Example Scene

CyberithIK comes with a prebuilt example scene to demonstrate the packages capabilities. You can find it under **CybSDK/IKSystem/Examples/Cyberith IK Scene**.

5. Add Layers and Tags

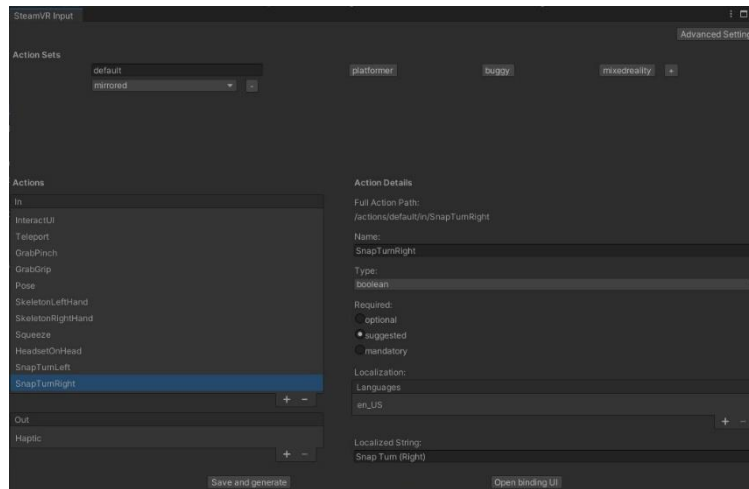
The CyberithIK Plugin requires the layers “LocalPlayer” and “RemotePlayer”. As mentioned [before](#), the layers are required for the 3rd person view, that’s shown in the mirror of the sample scenes. Importantly, multi user applications also require the use of both layers.



If you have not added the layers or tags yet, select the LayerAdder prefab which you can find in the folder **CybSDK/IKSystem/Addons/Layer Adder**. In the inspector, press “Add Layers” and “Add Tags”. The Unity console will confirm the creation of the layers “LocalPlayer” and “RemotePlayer” and the tag “Observer”.

6. Generate Input Bindings for SteamVR

The SteamVR Unity plugin requires input bindings to be generated to be able to utilize the controllers and trackers. You can generate the input bindings by opening the **SteamVR Input** menu, which can be found under the “Window” menu. You can skip this step if you have generated them already.



7. Start your 4-point CyberithIK experience

Now, you should be ready and set up to try out the 4-point Cyberith IK Scene with your Virtualizer, your HMD and 2 VR controllers.

Please go through the calibration screen by making a T-Pose and pressing the trigger while you are standing in T-Pose and looking straight forwards.

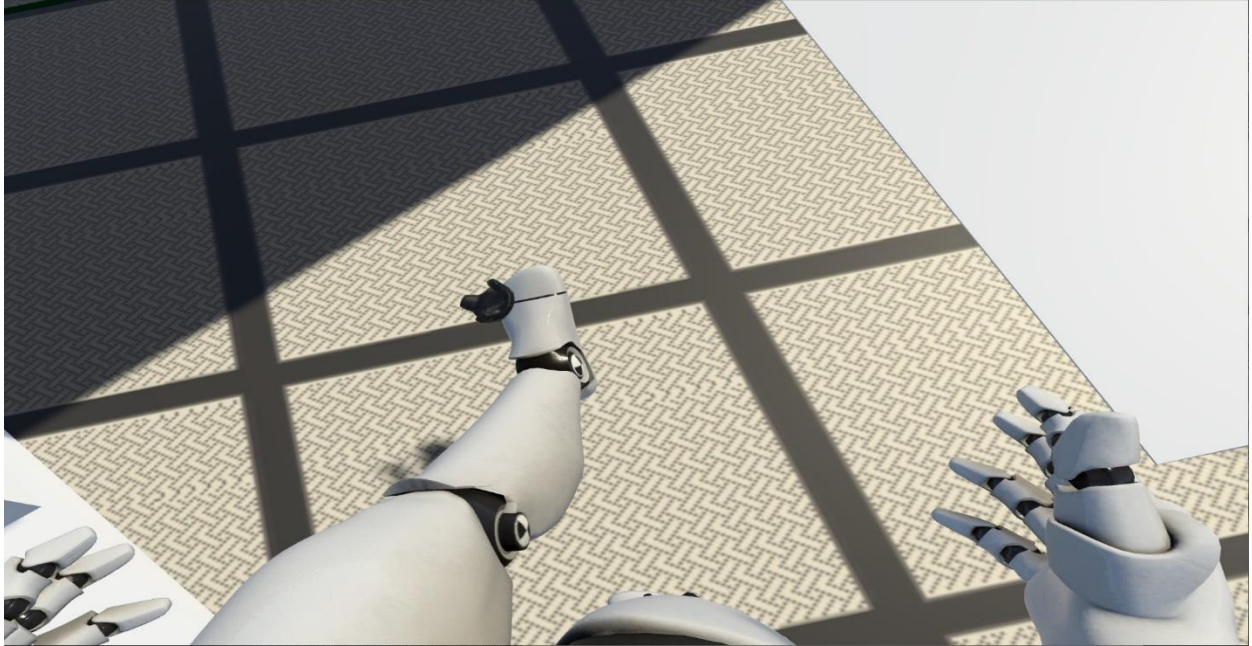
Please specifically check:

- The movement of your avatar, visible in the mirror: Look in the mirror and try different postures. Your Avatar should represent them accordingly.
- The procedural animation of walking: Walk on your Virtualizer to see the animated legs visible in the mirror.

8. Start your 6-point CyberithIK experience

Mount your SteamVR Trackers as instructed in the according document, turn them on and make sure the trackers are set to the roles “Left Foot” and “Right Foot”. Then, just re-start the experience. The Vive Trackers should be detected automatically and CyberithIK should launch in the 6-point IK mode automatically.

Note: In case that does not work, please check if the SteamVR bindings for both the “Tracker on Left Foot” and the “Tracker on Right Foot” are connected accordingly. You can access the bindings by going to the SteamVR menu and then clicking on “Manage Controller Bindings” while the application is currently running.



If 6-point IK is used, you can look down to see your feet and legs moving, according to the movement of your real-world feet.

Example Project using Microsoft Rocketbox Avatar

After testing the basic sample scene that's using the Cyberith Avatar, you can also use test the example scene that's using a Microsoft Rocketbox Avatar. You can find it in ***CybSDK / IkSystem / Examples / Cyberith IK Rocketbox Scene***

Conceptually, it works exactly as it does in the basic sample scene. The Layers and Tags only need to be added ("LayerAdder") if they weren't added before, when using the other scene.

Setup Your Own Project

If you want to implement the CyberithIK system in your own scenes, you can follow the following steps:

1. Import the Cyberith Core Plugin for Unity and the CyberithIK Plugin in your project

At the time you are starting to implement CyberithIK, you should be familiar with the Cyberith Core SDK (CybSDK) Plugin for Unity already. If you are not familiar with the CybSDK yet, please go through the applicable documentation, the [Unity Core Integration Guideline](#).

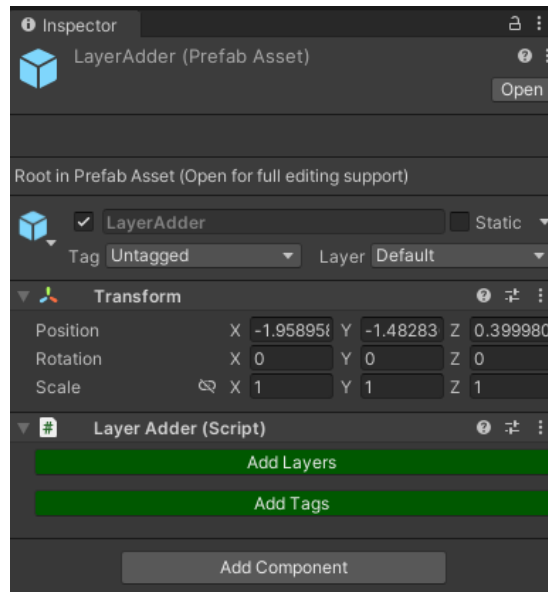
2. Import the SteamVR Plugin

CyberithIK is dependent on the SteamVR Plugin. Please import the SteamVR Plugin, that you can find (for free) on the Unity Asset Store: <https://assetstore.unity.com/packages/tools/integration/steamvr-plugin-32647>

3. Add Layers and Tags

CybIKSDK requires the layers “LocalPlayer” and “RemotePlayer”. As mentioned [before](#), the layers are required for the mirror, that’s integrated in the sample scene. Importantly, multi user applications also require the use of both layers.

If you have not added the layers or tags yet, select the LayerAdder prefab which you can find in the folder **CybSDK/IKSystem/Addons/Layer Adder**. In the inspector, press “Add Layers” and “Add Tags”. The Unity console will confirm the creation of the layers “LocalPlayer” and “RemotePlayer” and the tag “Observer”.



4. Open your Scene

Open the scene in which you want to implement the Virtualizer and CyberithIK system.

5. Add CVirtPlayerControllerIK Prefab to Scene

The CVirtPlayerControllerIK prefab contains all the scripts for the Virtualizer and the CyberithIK system. This prefab has the Cyberith avatar included and does not require any further initial setup to get started. The CVirtPlayerControllerIK prefab can be found in **CybSDK/IKSystem/Core/Prefabs**.

Note: If you are using different hardware than the HTC Vive Pro or the HTC Vive Controllers, then follow the links to go to the instructions to adjust [the head of the model](#) or [offset other parts of the model](#).

Note: If you want to use your own model or one of the Microsoft Rocketbox avatars, you need to link the right components to each other in the inspector in Unity. Instructions on how to setup a Rocketbox Avatar or your own model can be found in the following [chapter](#).

6. Start your 4-point CyberithIK experience

Now, you should be ready and set up to try out the 4-point Cyberith IK Scene with your Virtualizer, your HMD and 2 VR controllers.

Please go through the calibrations screen, by making a T-Pose and pressing the trigger while you are standing in T-Pose and looking straight forwards.

Please specifically check:

- The movement of your avatar, visible in the mirror: Look in the mirror and try different postures. Your Avatar should represent them accordingly.
- The procedural animation of walking: Walk on your Virtualizer to see the animated legs visible in the mirror.

7. Start your 6-point CyberithIK experience

Mount your SteamVR Trackers as instructed in the according document, turn them on and make sure the trackers are set to the roles “Left Foot” and “Right Foot” in the SteamVR settings. Then, just re-start the experience. The Vive Trackers should be detected automatically and CyberithIK should launch in the 6-point IK mode automatically.

Setup Custom Avatar in Your Own Scene

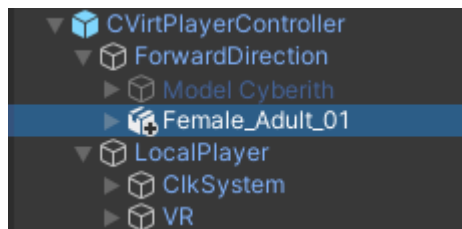
You might want to use a different model than our Cyberith avatar. You can use the Microsoft Rocketbox avatars that we discussed [previously](#) or add a model from a different source. Before you add a different avatar, make sure to have followed the steps of the previous chapter.

1. Add the Model Files to Your Project

Import the model files of your avatar. For example, each Rocketbox avatar has the folders “Export”, containing the FBX file, and “Textures”.

2. Insert the Model into the Forward Direction Game Object

The FBX file of your model should be added to your scene as a child of the ForwardDirection game object that is part of the CVirtPlayerControllerIK prefab. Make sure that all the values of the position and rotation of the Transform of your model are at 0.



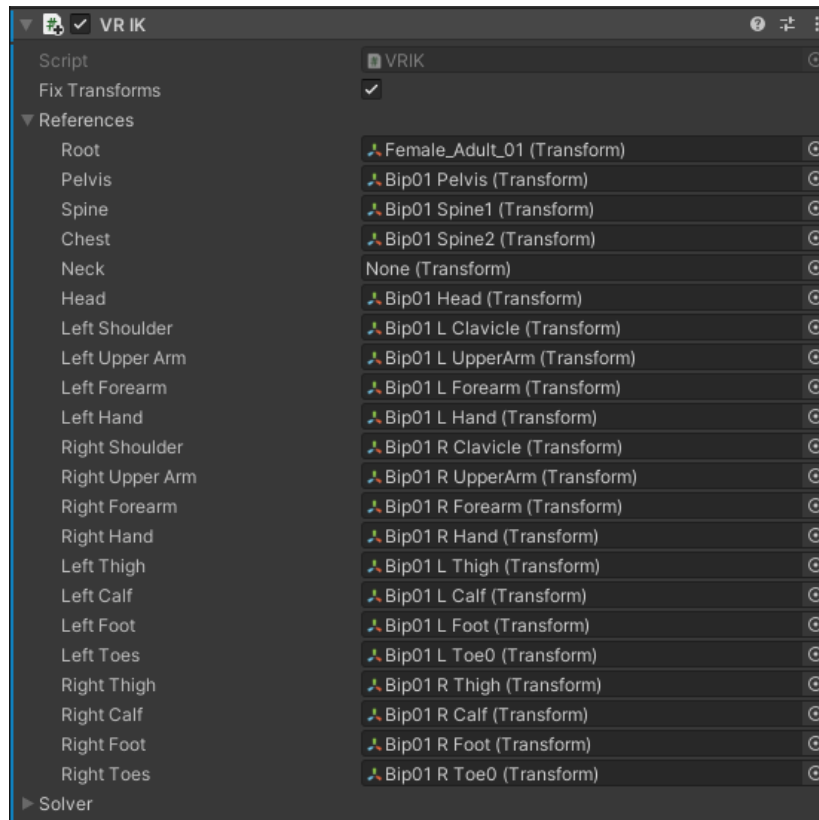
3. Copy Components to Avatar Game Object

Copy the following components from the “Model Cyberith” to the game object of your own model: **VRIK**, **ClkSystemCalibratedModel** and **ClkFingerSystem**. To copy, right-click the script in the Model Cyberith object and press “Copy Component”. Go to the game object of your model and right-click the top bar of the Transform component and press “Paste Component As New”.

4. Add References in VRIK Component

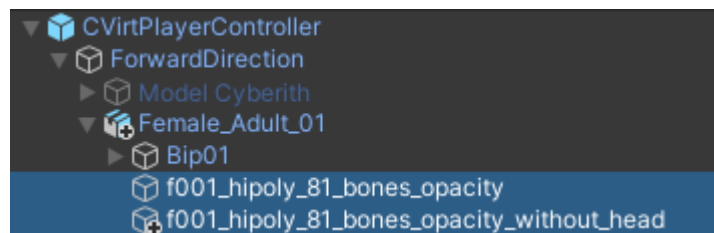
Within the VRIK component of your model, you have to add the references to the game objects of the different body parts.

Important: Leave the “Neck” reference empty!

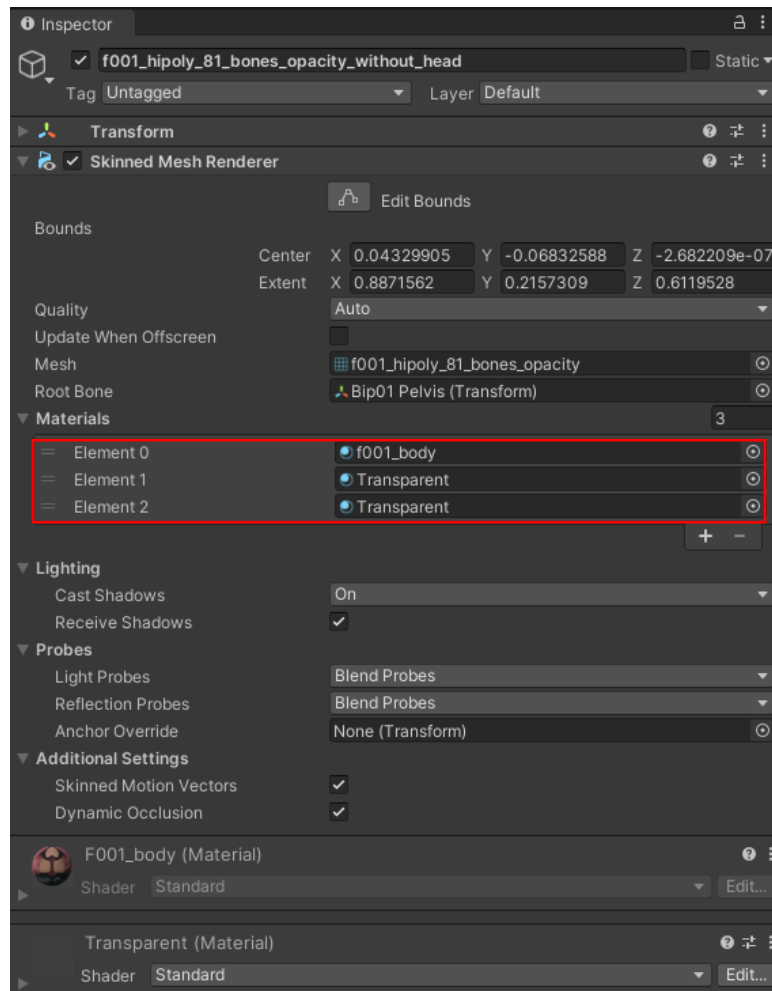


5. Duplicate Skinned Mesh Renderer Game Object (Necessary for Rocketbox)

The Cyberith Avatar has its mesh renderers separated in different game objects. This enables us to make the head invisible in first-person in the next step. The Rocketbox models, however, have only one game object for the mesh renderer. The solution is to duplicate the GameObject with the Skinned Mesh Renderer. This step is not required if your avatar has the mesh renderers separated already, like the Model Cyberith.

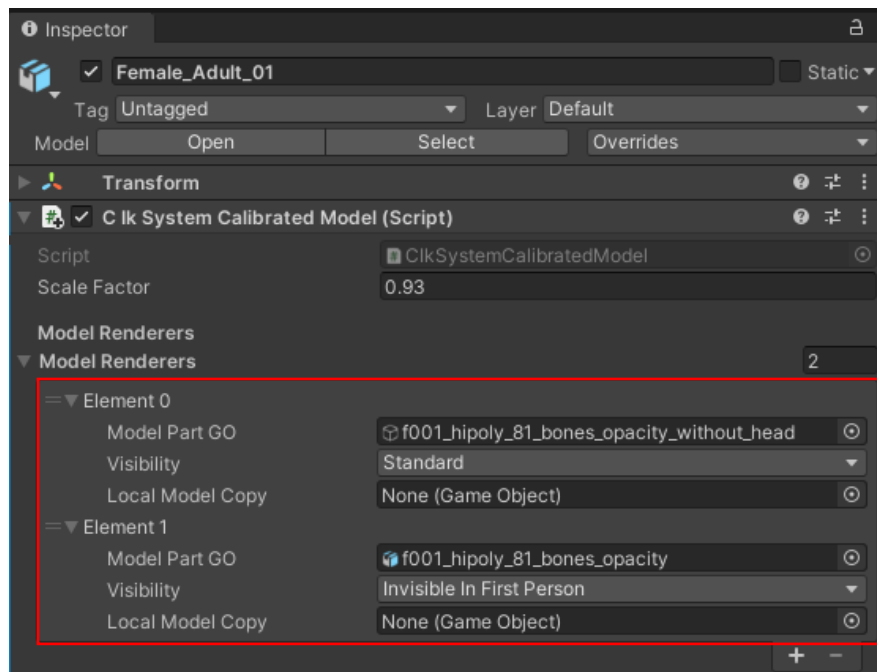


After duplicating, you have to apply the “Transparent” material to the head. The material can be found in the folder CybSDK/IKSystem/Addons/ClippingPlane. You can either drag and drop the material onto the head or place it in the material section of the Skinned Mesh Renderer component.



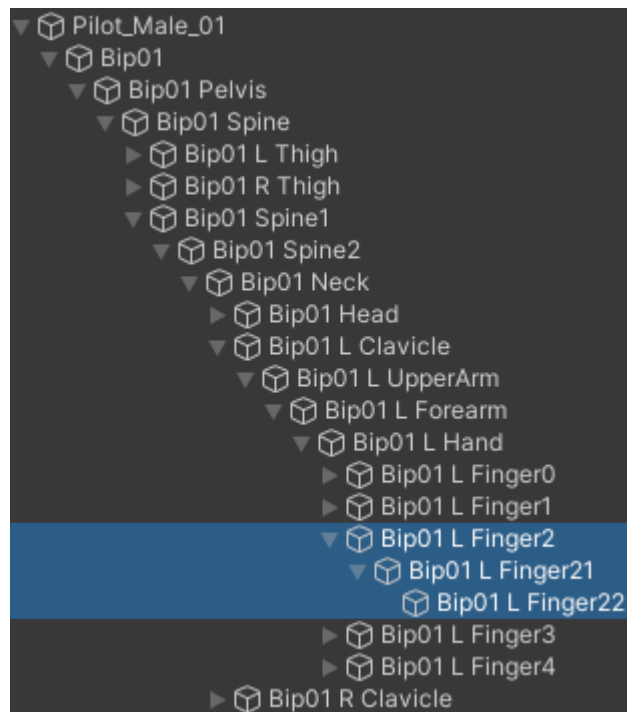
6. Add the Correct Model Renderers

In the **C Ik System Calibrated Model** component you can add the right Skinned Mesh Renderers and set if they are visible to all (“Standard”) or invisible in first-person. In the field **Model Renderers**, remove the objects that belong to the Model Cyberith and add the GameObjects with the Skinned Mesh Renderers of your avatar. Set the visibility of the GameObjects that you would like to hide in first-person (e.g., the head) to “Invisible in First-Person”. For the Rocketbox Avatars, this means hiding the full model, while setting the model without head to “Standard”:

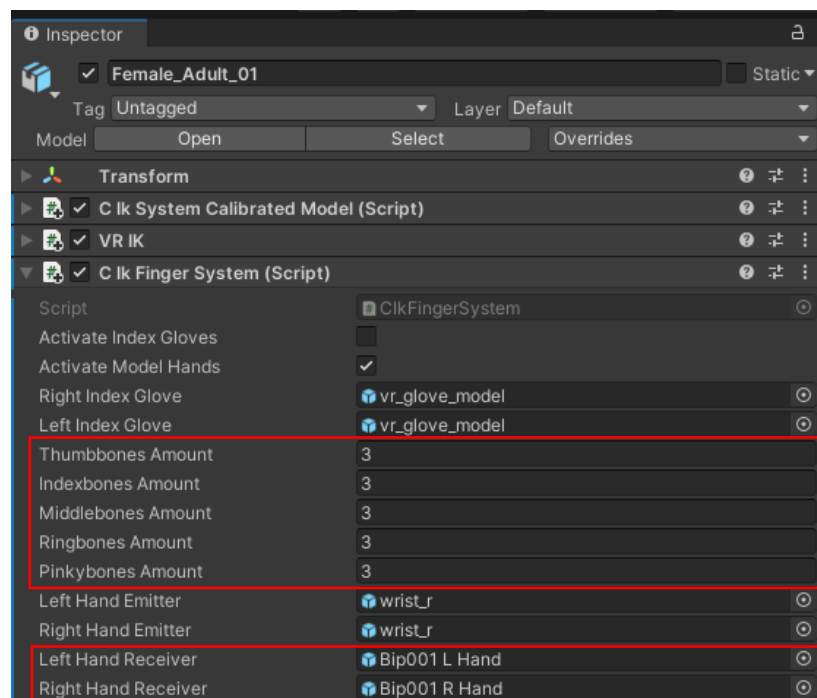


7. Set up C Ik Finger System Component

If you use a Rocketbox Avatar, you need to change number of bones set in the **C Ik Finger System** component (e.g., change the field “Thumbbones Quantity”). For the Rocketbox Avatars **the number of finger bones is 3**. This number might be different if you use a custom model. You can find the number of bones by looking at the bone structure in your scene Hierarchy:



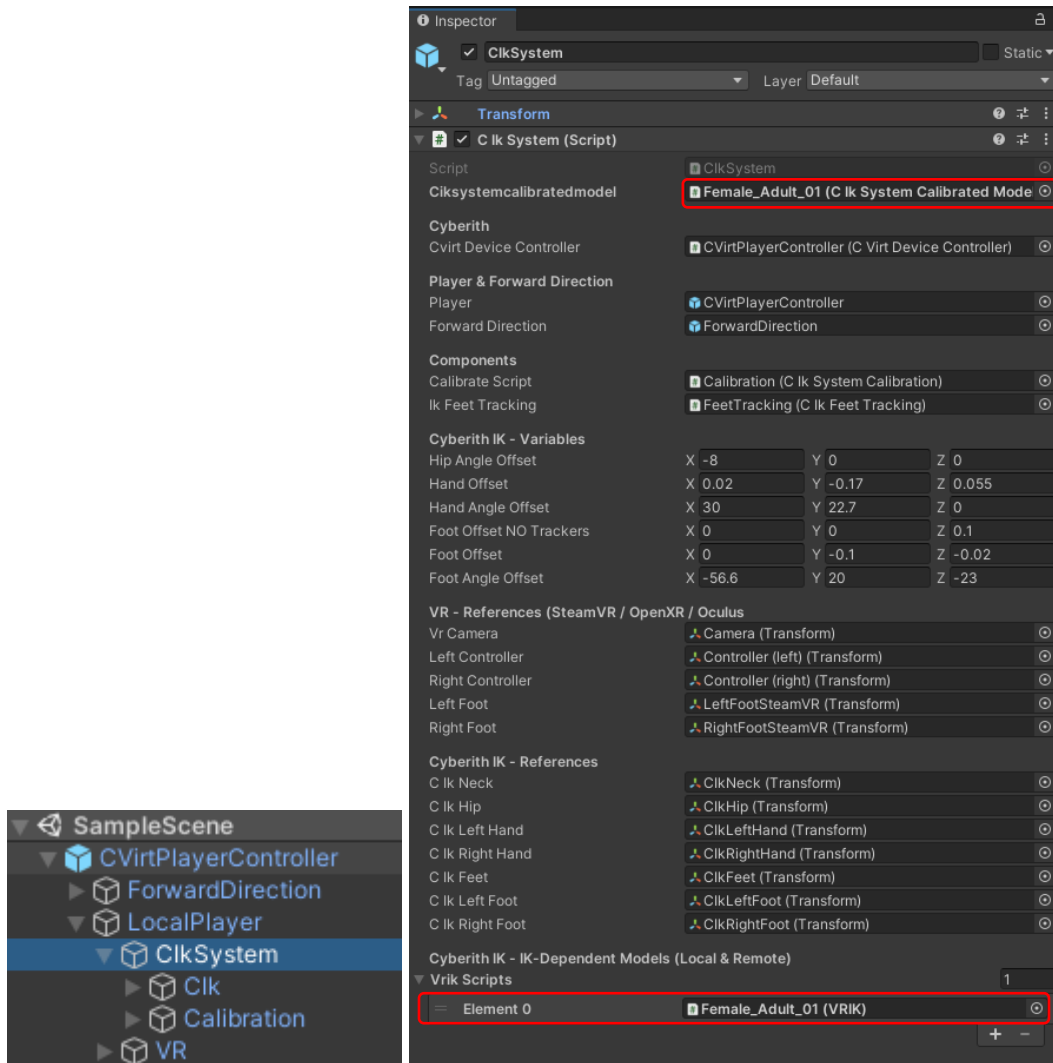
Next to setting the number of bones, you need to reference the hands of your new model in the “Left Hand Receiver” and “Right Hand Receiver” field.



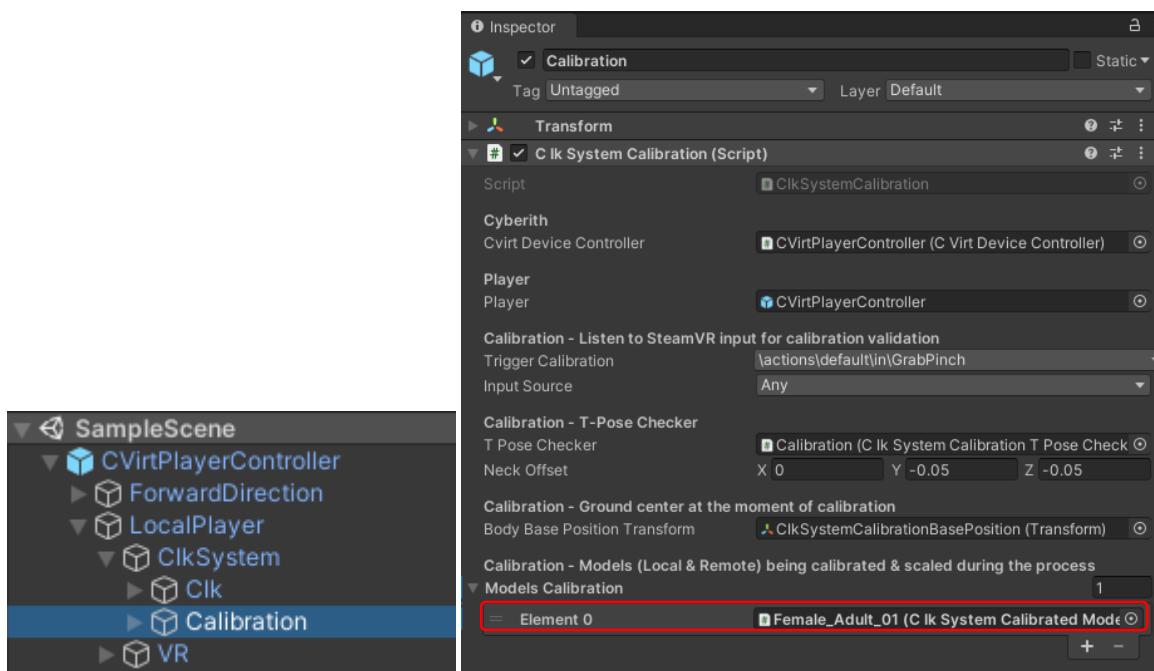
More information on this script can be found in a later [chapter](#).

8. Add the References to Your Avatar

Several components need a reference to your avatar. Drag and drop the GameObject of your avatar into the fields **Ciksystemcalibratedmodel** and **Vrik Scripts** of the C Ik System component of the CIKSystem GameObject.

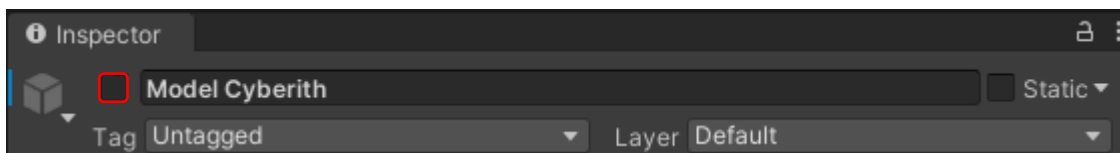


Then drag and drop your avatar in the field **Models Calibration** of the C Ik System Calibration of the Calibration GameObject.



9. Disable Cyberith Model Game Object

Select the GameObject of the Cyberith Model and disable it in the Inspector.



10. Test Your Avatar

Test if your avatar has been correctly set up. If any of your body parts seem incorrectly calibrated, you might want to look again at all the steps discussed in this chapter.

Documentation

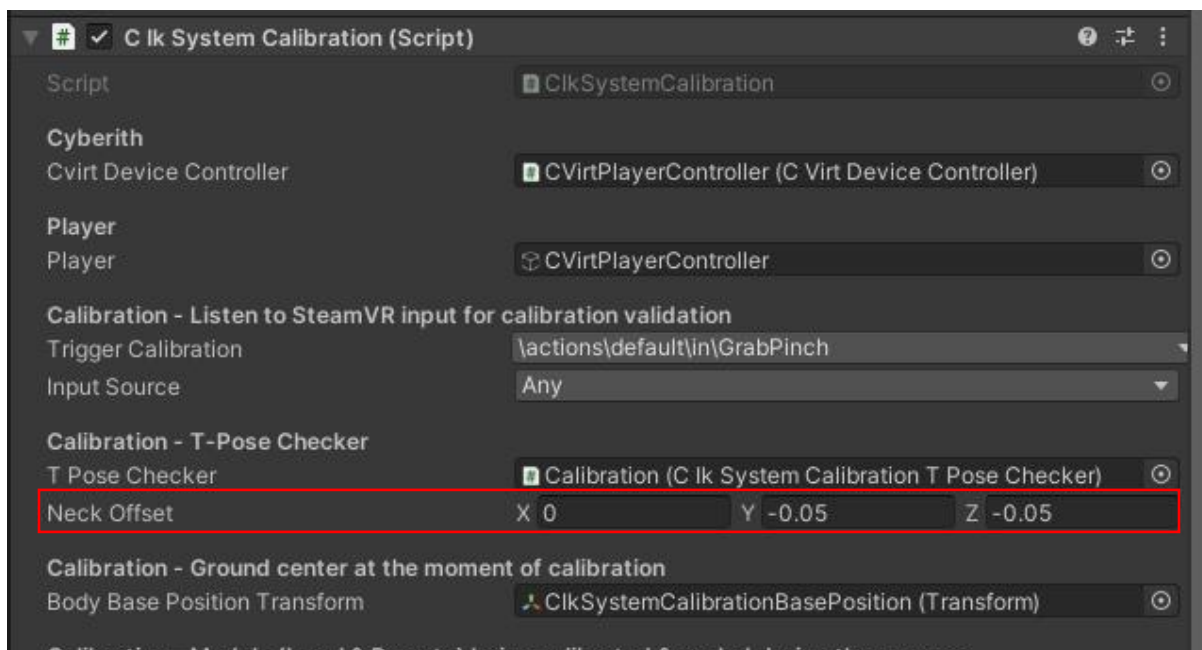
The C Ik System Calibration Script & the Neck Offset

The **Cyberith IK System Calibration Script** handles the initial calibration and provides required values to the **C Ik System Script**.

You can find it in “CvirtPlayerController->LocalPlayer->ClkSystem->Calibration”



Very importantly, the C Ik System Script contains the **Neck Offset** (as explained above). The Neck Offset is the central and most important offset that can be adjusted according to the hardware used. **For both the HTC Vive Pro & the Valve Index we found the value (0,-0.05,-0.05) to be suitable.**



In order to **optimize the Neck Offset for your VR headset, please try out different values**. You can increase and decrease values in a step by step way to figure out which values work best!

As **the Neck Offset is defining the central axis that you are turning around**, you should optimize the value when facing in multiple different directions.

It makes sense to turn by 90° 3 times, so that you test the same Neck Offset in both the forwards and the backwards direction of both the X and the Y axis:

- In case the Y-Offset of the Neck Offset is a too large negative number:
 - The hips of your avatar will be too much in the back of you (behind your real hips), when you are looking forwards.
 - In turn, the hips of your avatar will be too much in front of you, after turning 180 degrees.
 - In case you turn 90° to the left or to the right, the hips will be next to you (next to your real hips).
 - Of course the opposite effects happen if the Y-value of your Neck Offset is a too low negative number (or even a positive number).
 - To better understand the concept, you can try to increase the Y-value to a very high one (e.g., -0.5) to better visualize what's happening.
- In case the Z-Offset of the Neck Offset is set to a wrong number, there will be issues related to the height of your avatar and the scaling of the all bones and offsets.

Important:

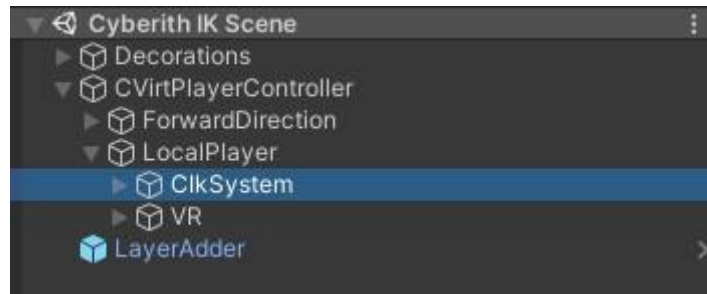
The Neck Offset is used in the calibration phase and is thus in the calibration script.

To optimize the Neck Offset, you will need to re-calibrate multiple times and check if your avatar is positioned correctly.

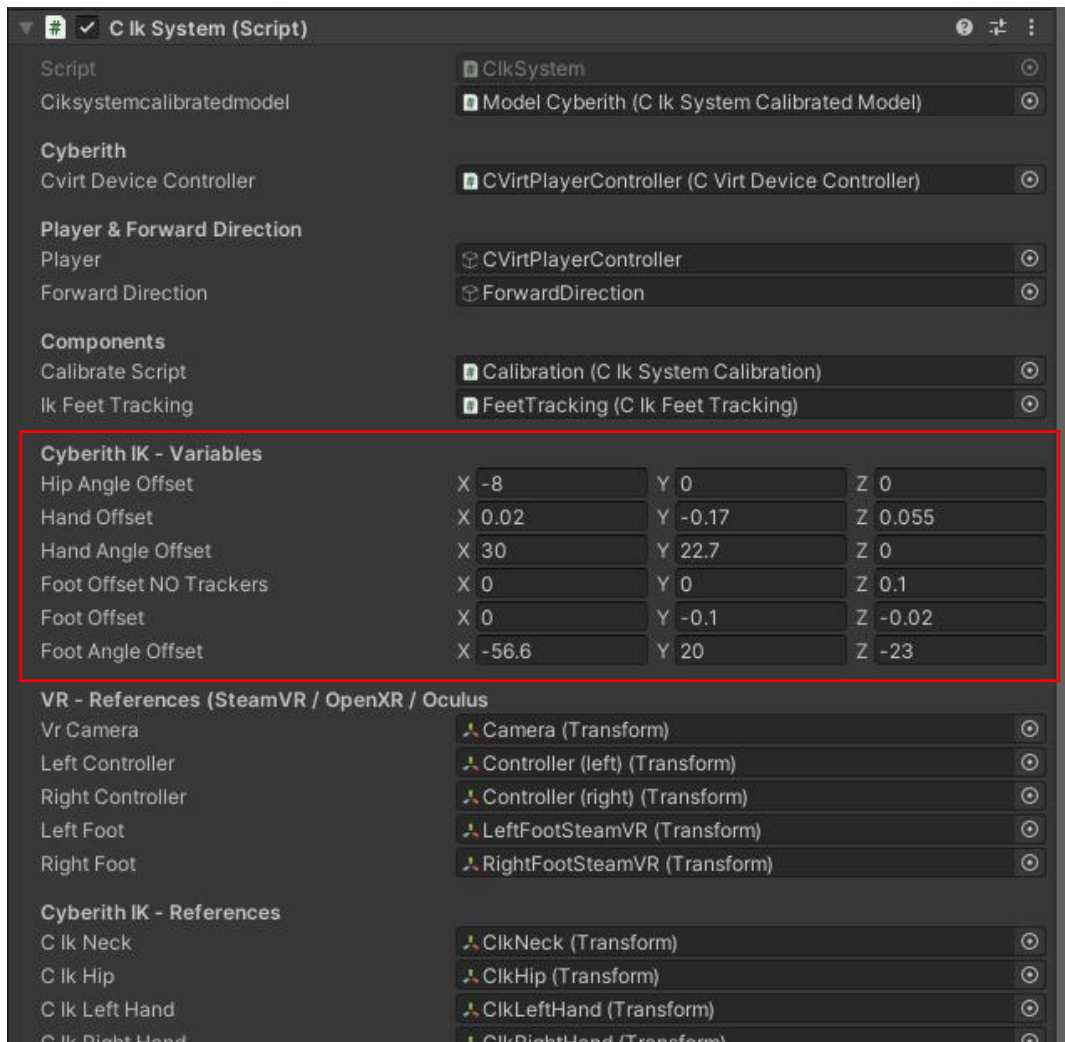
A **second phase of the calibration adjusts the length of the arms**. This second phase of calibration also uses the hand offset from the C Ik System Script as laid out in the next chapter.

The C IK System Script and the remaining Offsets

The **Cyberith IK System Script** and handles the calculations of CyberithIK in real time. You can find it in “CvirtPlayerController->LocalPlayer->ClkSystem”.



The C Ik System script contains all the other offsets, as explained in the introductory chapter above. All of these offset values are best optimized by trying out different values.



Please note that the optimal values may depend both on the Hardware and the Avatar Models used.

When it comes to optimizing the offsets, there is a difference in between the Hand Offset and the remaining offsets.

The Hand Offset is used both in the calibration phase and in runtime thereafter.

To optimize the Hand Offset, please follow this procedure:

1. Optimize the Hand Offset in runtime (after calibration). Adjust the values in a way, so that the position of the wrist of your avatar (your virtual wrist) matches precisely with the position of your real wrist. (In this stage, don't look at the length of your arms.)
2. Re-calibrate with the new values and check if the calibration fixed the length of your arms correctly (and if your virtual wrist is still at the correct position).

Note: While one may argue that the Hand Offset should be moved to the C Ik Calibration Script from a logical standpoint (because it is used in the calibration phase), this would make optimizing the value significantly harder (as it would not be possible to optimize the value in runtime). Thus, we decided to keep it in the C Ik System Script, as this is more practical.

All remaining offset values (incl. the Hand Angle Offset) are not used in the C Ik Calibration Script. Thus, you can adjust the remaining values at runtime without the need to re-calibrate!

In general, we recommend to use approx. the following values:

Note: This list has not been completed yet.

- **Hand Offset:**

- HTC Vive Controllers (“Vive Wands”): **0.02, -0.17, 0.055**
- Valve Index Controllers (“Knuckles”):

- **Hand Angle Offset:**

- HTC Vive Controllers: **30, 22.7, 0**
- Valve Index Controllers:

- **Hip Angle Offset:**

This offset may need to be adjusted if a different avatar model is used. If the hip is standing out to the front or too the back too much by having an angle (looks a little bit like a bee), you should optimize the Hip Angle Offset. Usually, **only the x-value** of this offset needs to be adjusted!

- Cyberith Avatar: **-8,0,0**
- Microsoft Rocketbox Avatar:
- FinalIK Crashtest Dummy: **25,0,0**

- **Foot Offset NO Trackers:**

Only used if trackers are not used (4-point IK). Adjusts both feet together.

- Generally recommended value: **0,0,0.1**

- **Foot Offset:**

Only used if trackers are used (6-point IK).

- Vive Trackers mounted exactly as explained above: **0,-0.1,-0.02**

- **Foot Angle Offset:**

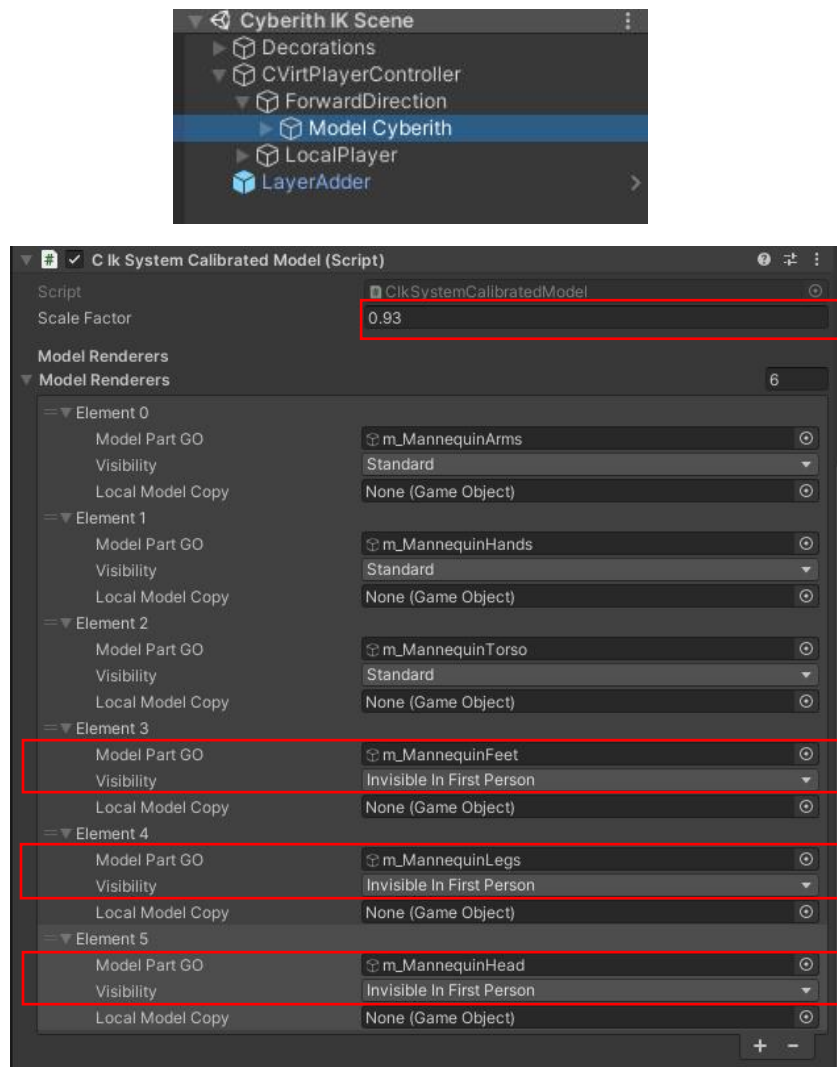
Only used if trackers are used (6-point IK).

- Vive Trackers mounted as explained above on the Cyberith Avatar: **-56.6,20,-23**
- Vive Trackers mounted as explained above on the Rocketbox Avatar:

The C Ik System Calibrated Model Script: Materials and Mesh Renderers

Aside of handling the scaling, the “**C Ik System Calibrated Model**” script is also responsible for managing the visibility of the different parts of the body through the layers explained above.

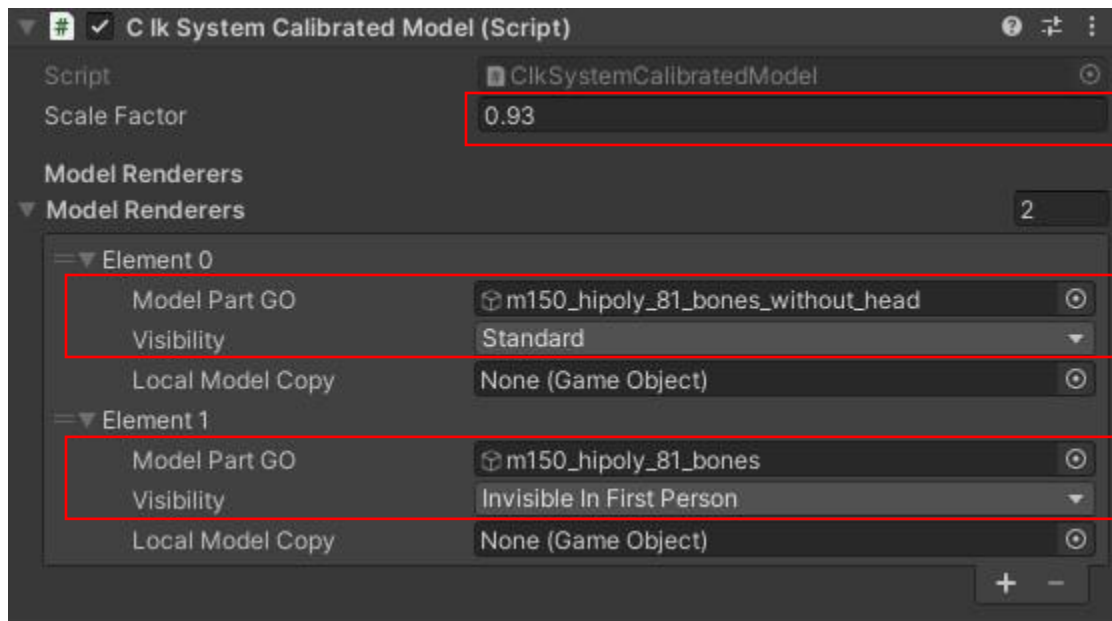
You can find it in the model you are using, so, for example in “*CvirtPlayerController -> ForwardDirection -> Model Cyberith*” or in “*CvirtPlayerController -> ForwardDirection -> Pilot_Male_01*”.



For the Cyberith Avatar, the feet, the legs & the head is set to invisible in 1st person view, while they remain visible in the 3rd person view.

The other parts of the body remain visible in all views/layers (setting called “Standard”).

Note: Do not touch the scale factor of “0.93”. This value comes from FinalIK and must remain that way!



The Rocketbox Avatar does not come with separate mesh renderers for the feet and the legs. It only separates in between the complete model and one without the head.

While the model without the head is set to standard (visible on all layers), the model with head is set to "Invisible In First Person" to avoid the user being able to see into the head from the inside.

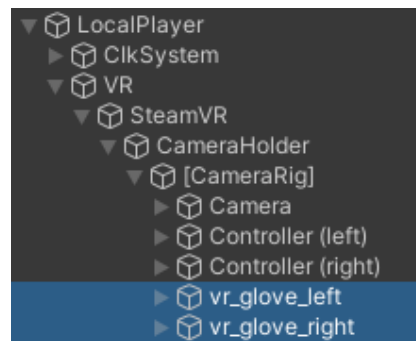
Note: Exactly like for the Cyberith Avatar, the scale factor is set to "0.93". **Do not change it!** This value comes from FinalIK and must remain that way!

Add-On Feature: The C Ik Finger System Script

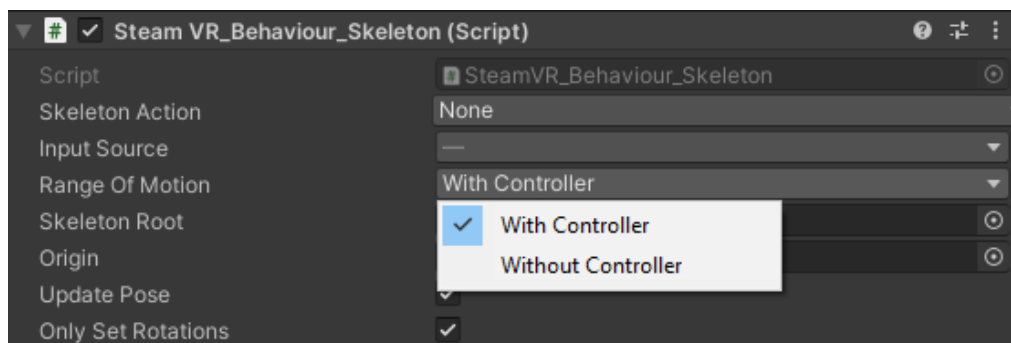
The **Cyberith IK Finger System Script** handles finger tracking and can be found in the model you are using, so for example in “CVirtPlayerController -> ForwardDirection -> Model Cyberith”.



This script takes the values from the SteamVR glove model, that's provided by Valve and applies the corresponding values to the fingers of the model you use! The SteamVR gloves can be found under the “[CameraRig]” GameObject:



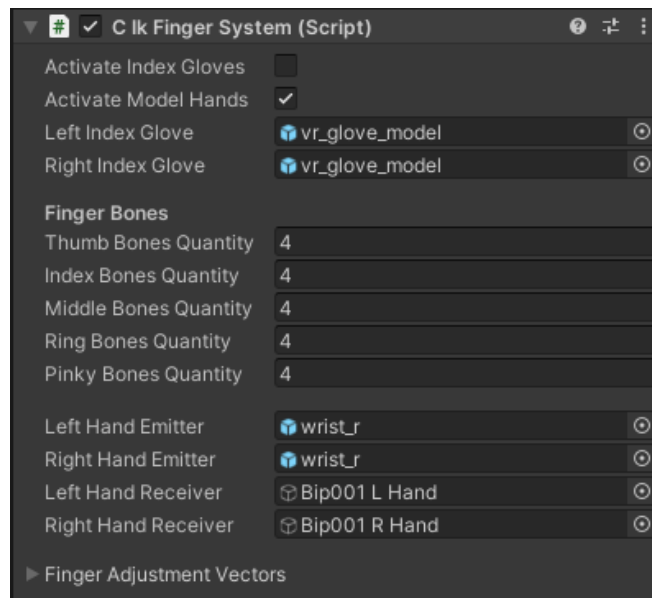
In the “**SteamVR_Behaviours_Skeleton**” component, you can set if the fingers should move “With Controller” or “Without Controller”. This means that your fingers are either wrapped around a controller when contracted, or they are not constrained by a controller and can thus make a closed fist.



Inspector Variables

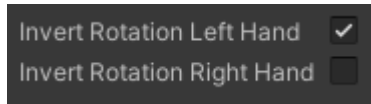
In the inspector you can adjust the following variables for debugging your model:

- **Activate Index Gloves**
 - Show the 3D model of the Index Gloves.
- **Activate Model Hands**
 - Show the hands of the used avatar.
- **Right Index Glove/Left Index Glove**
 - Object references to the `vr_glove_model` of the Index Gloves.
- **Bones Quantity**
 - Here you setup the number of bones of the hands of the used 3D avatar. (The **Cyberith Avatar** has **4 bones** at each finger while the **RocketBox Avatar** has **3 bones** in each finger.)
- **Right Hand Emitter/Left Hand Emitter**
 - Object references to `wrist_r` of the Index `vr_glove` bones where we grab the rotations of the bones.
- **Right Hand Receiver/Left Hand Receiver**
 - Object references to the hands of the bone system of the used Avatar.

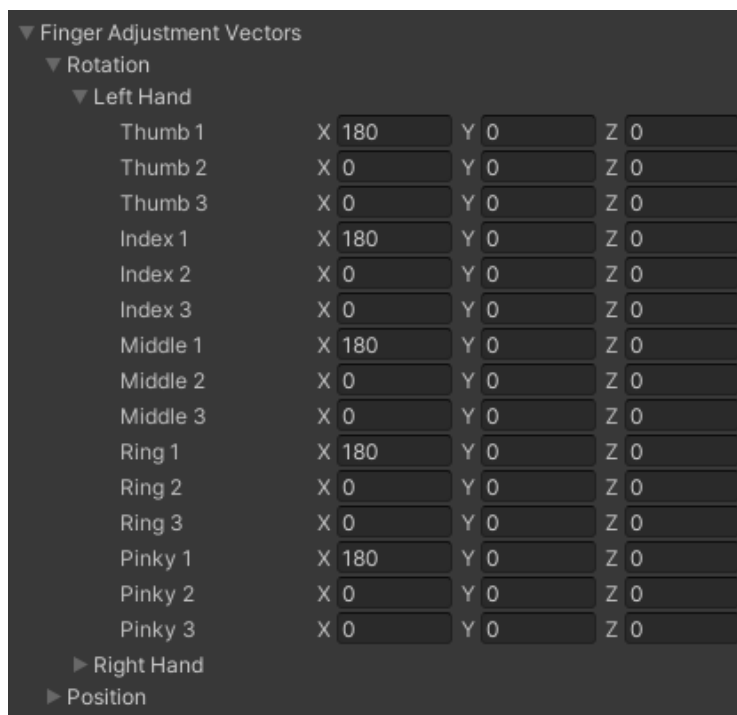


- **Finger rotations and positions**

- It might happen that on one hand the fingers do not move correctly when contracting. Instead of a fist, the fingers rotate upwards. This can be fixed by inverting the rotation of the hand.



- At last we can make adjustments for each finger bone at each hand. These changes are only visible during runtime. The positions are added to the initial positions of the fingers.
- While it may not be required to adjust the finger positions in most cases (according to our experience, maybe except of the thumb), **the finger rotations typically need to be adjusted** for every single finger.
- To adjust the finger rotations, adjust one finger after another.
- Start by adjusting the first joint of the finger (the joint closest to the arm), then go to the next joint closer to the tip of the finger until you reached the tip. A rotation in the upper joint, also rotates the lower (child) joints in the finger.



Changing XR Plug-in

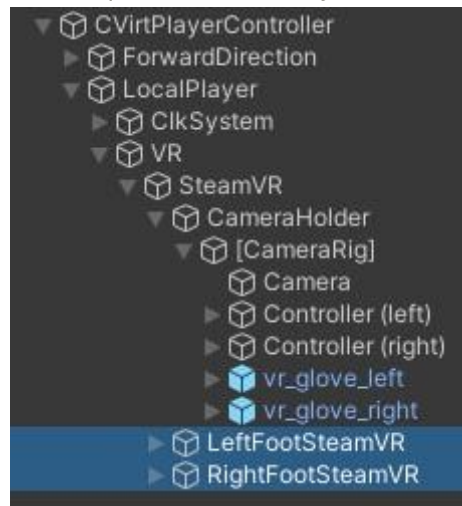
Although we recommend using the Steam VR Unity plug-in, you might have reasons to use a different XR plug-in. It is possible to use other XR plug-ins, for example, OpenXR. However, there are some steps required to make it work. Furthermore, **the C Ik Finger System script does not work without Steam VR**. As of writing this manual, OpenXR Unity package does not support the Valve Index finger tracking that the C Ik Finger System relies on.

To change to a different XR plug-in, please follow the following steps:

- 1) Exchange the VR camera rig (including controllers).



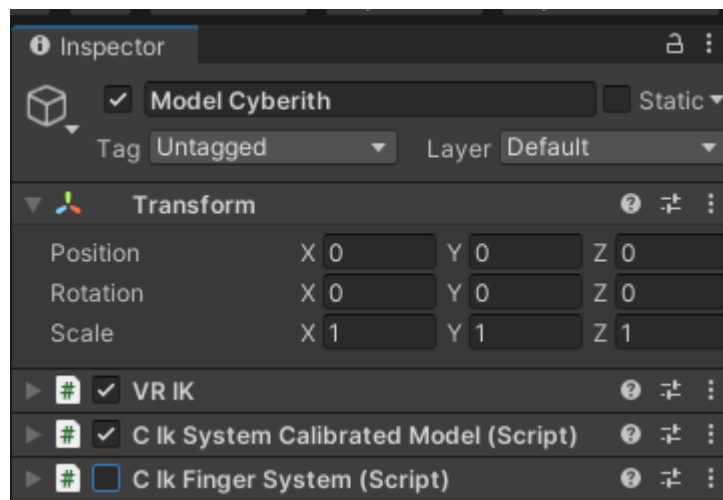
- 2) Add feet tracker input and exchange the tracker GameObjects of SteamVR (e.g., "LeftFootSteamVR") with their equivalent GameObjects of the new XR plug-in.



- 3) Reference the GameObjects of the camera, controllers and feet trackers in the C Ik System component in the ClkSystem GameObject.



- 4) Adjust scripts that reference SteamVR (Valve.VR).
 - a) *Core/Scene/Mirrors/Scripts/MirrorReflection.cs*: The HMD transform is retrieved through SteamVR.
 - b) *Core/Scripts/ClkSystemCalibration.cs*: SteamVR is used to calibrate after a button press.
- 5) Disable/remove C Ik Finger System component in your model.



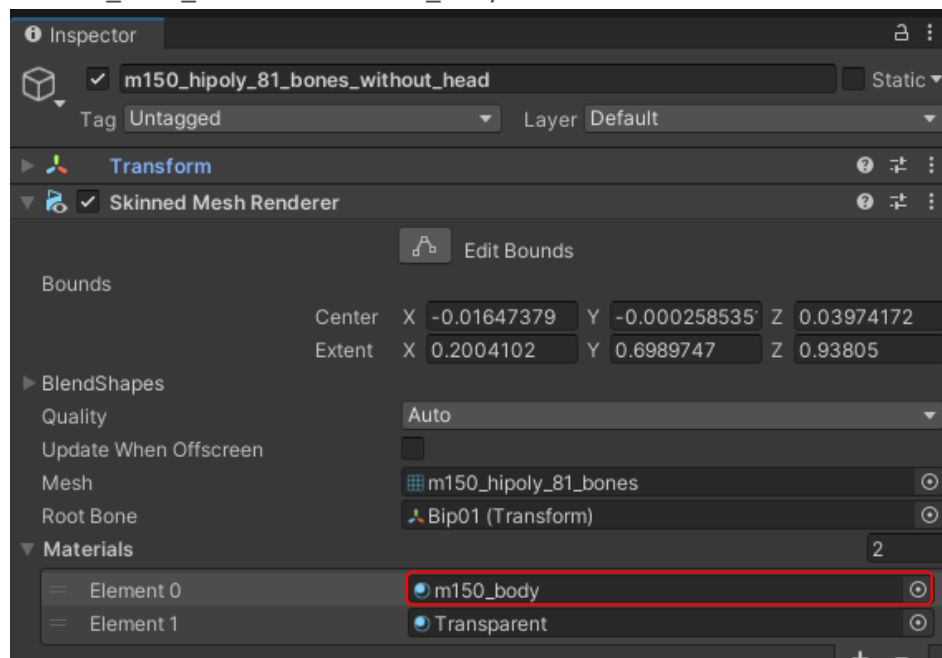
Experimental: Clipping Plane Shader

The idea of this experimental feature is to hide the legs and feet of an avatar, that does not have a separate mesh renderer for these parts of the body. A plane is attached to the user's hips (pelvis) and all parts of the body that are located below that plane are made transparent / invisible.

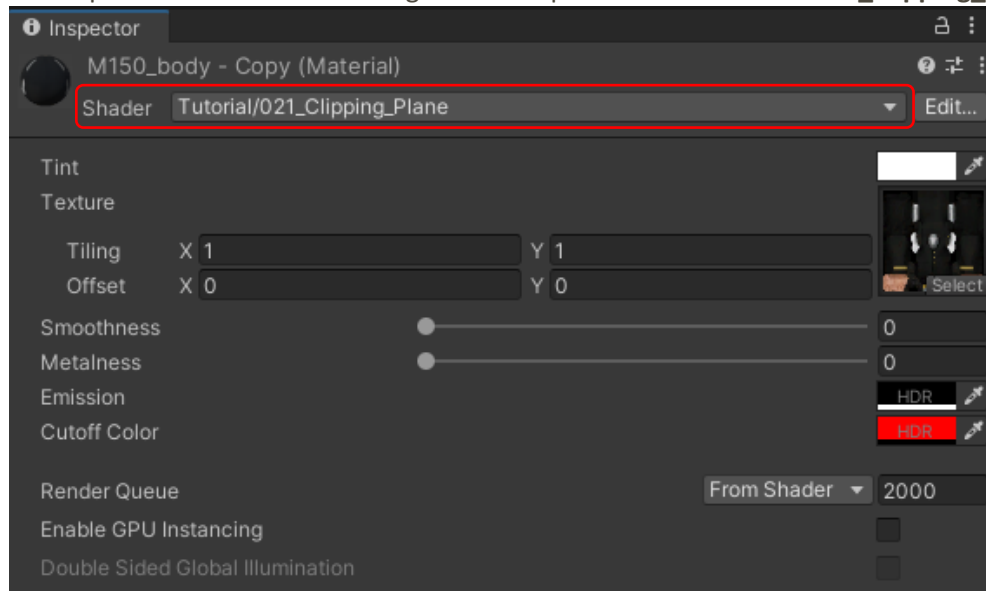
This feature is considered experimental, because it comes with the problem that other parts of the body are also hidden when they get below the height of the plane. This is happening easily for the hands, as people move their hands below the height of their hips regularly.

Follow these steps to experiment with the Clipping Plane Shader:

- 1.) Find in your project file structure the body material file that is attached to the Skinned Mesh Renderer of your avatar and duplicate it. For example, the body material of the Rocketbox Avatar "Pilot_Male_01" is called "m150_body".

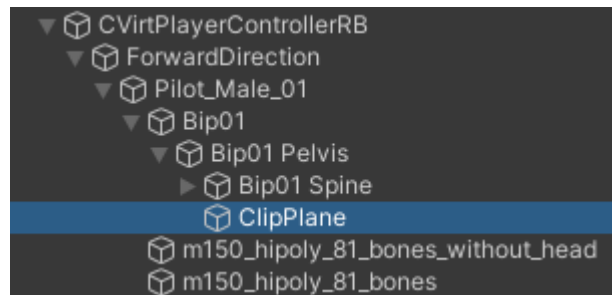


- 2.) Select the duplicated material and change in the Inspector the shader to “021_Clipping_Plane”.

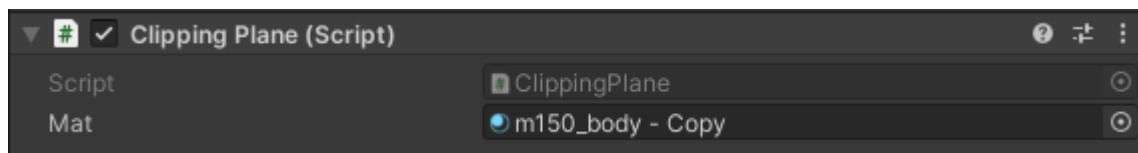


In case, the Texture field is empty, add the relevant texture of the material.

- 3.) Attach a plane to the hips of the Rocketbox Avatar, so that the plane moves up and down with the hips (e.g., when the user crouches down).



- 4.) Disable the Mesh Collider and Mesh Renderer of the plane you just created.
- 5.) Add the script “Clipping Plane” to the created plane and reference the material you duplicated in the field “Mat”.



- 6.) When playing your scene, your avatar should look like the following after calibrating:

